

Exactness of forward-table merge-cell pruning (Fix 2) for `extract_cell`

network-matching — cell-DAG extraction, §8.2

2026-07-17

Claim

Before running `extract_cell`, prune every *merge* candidate cell whose forward-table value exceeds the cost of a known valid matching. The pruned run returns the *same* optimal matching, bit-for-bit, as the un-pruned run. Formally: with lower bound $\text{LB}(m, v) = D[m][v]$ (the forward table alone), upper bound $\text{UB} = C(M_J)$ for an incumbent matching M_J from the `extract_join` engine, and pruning restricted to merge cells ($\text{indeg}(m) \geq 2$), the result is exact.

Setup and notation

Let A and B be the source and target DAGs. For an A -vertex a , write $\text{Apred}(a)$ for its parents and $\text{outdeg}(p)$ for the number of children of p in A .

- $\mathcal{M}$ is the set of **valid matchings** (satisfying V1–V3 and the run/move structure).
- For $M \in \mathcal{M}$, $\text{run}_M(a) \subseteq V(B)$ is the run assigned to a . A **cell** (a, v) is *used by* M iff $v \in \text{run}_M(a)$.
- Cost

$$C(M) = \sum_a \sum_{v \in \text{run}_M(a)} w_M(a, v) E(a, v),$$

where $E(a, v) \geq 0$ and every ledger weight $w_M(a, v) \in \{1, \beta, \alpha\}$ with $\beta \geq 1$, $\alpha \in (0, 1]$ — all strictly positive. Hence **every summand is ≥ 0** (★)

and deleting any subset of summands cannot increase C .

- $c^* = \min_{M \in \mathcal{M}} C(M)$ and M^* denotes an optimiser.

The forward table. By the construction of `forward()` (main doc §4.1), $D[a][v]$ is a min-sum dynamic program; explicitly,

$$D[a][v] = \min_W \Phi(W),$$

the minimum over the legal **upstream configurations** W of (a, v) — an assignment of one cell to a and to every ancestor of a , move-consistent and V3-consistent, with a 's run *ending* at v — where

$$\Phi(W) = \sum_{(b,x) \in W} \gamma_W(b) w_W(b, x) E(b, x), \quad \gamma_W(b) = \prod_{e \text{ on the } A\text{-path } b \rightarrow a} \frac{1}{\text{outdeg}(\text{tail } e)} \in (0, 1].$$

The $1/\text{outdeg}$ factors are the split-sharing of §4.1; §6.1 exhibits that they *under*-count.

Proof

Proposition 1 (forward under-count). *For every $M \in \mathcal{M}$ and every cell (a, v) used by M , $D[a][v] \leq C(M)$.*

Proof. From M build the configuration $\widehat{M}$: keep M 's cell-assignment on a and all ancestors of a ; *truncate* $\text{run}_M(a)$ to end at v (discard cells of a 's run after v); and *discard* M 's assignment on all descendants of a .

- (i) $\widehat{M}$ is a legal upstream configuration for (a, v) : every move in it is a move of M (hence valid), it is V3-consistent because M is, and a 's run ends at v . As a feasible point of the minimisation defining $D[a][v]$,

$$D[a][v] \leq \Phi(\widehat{M}).$$

- (ii) Each $\gamma_{\widehat{M}}(b) \leq 1$ (a product of factors ≤ 1) and each $w E \geq 0$, so

$$\Phi(\widehat{M}) = \sum \gamma w E \leq \sum w E.$$

- (iii) The summands $\sum w E$ of $\widehat{M}$ are a *subset* of the summands of $C(M)$ (truncation only deletes summands; each kept cell retains its M -weight), and all are ≥ 0 , so by $(\star)$, $\sum w E \leq C(M)$. Chaining, $D[a][v] \leq \Phi(\widehat{M}) \leq C(M)$. $\square$

Proposition 2 (incumbent bound). $c^* \leq \text{UB}$.

Proof. $\text{UB} = C(M_J)$ for the incumbent $M_J \in \mathcal{M}$, and c^* is the minimum of C over $\mathcal{M} \ni M_J$. $\square$

Theorem 1 (exact prune). *Let*

$$\text{blocked} = \{(m, v) : \text{indeg}(m) \geq 2, D[m][v] < \infty, D[m][v] > \text{UB}\}, \quad \mathcal{M}' = \{M \in \mathcal{M} : M \text{ uses no blocked cell}\}.$$

Then $\min_{\mathcal{M}'} C = c^$, attained by M^* , and $\mathcal{M}' \neq \emptyset$.*

Proof. **(1) No optimal cell is blocked.** Let (m, v) be any cell used by M^* . By Proposition 1, $D[m][v] \leq C(M^*) = c^*$; by Proposition 2, $c^* \leq \text{UB}$. Hence $D[m][v] \leq \text{UB}$, i.e. $\neg(D[m][v] > \text{UB})$, so $(m, v) \notin \text{blocked}$. Therefore $\text{cells}(M^*) \cap \text{blocked} = \emptyset$, i.e. $M^* \in \mathcal{M}'$.

(2) Optimum preserved. $M^* \in \mathcal{M}'$ gives $\min_{\mathcal{M}'} C \leq C(M^*) = c^*$; and $\mathcal{M}' \subseteq \mathcal{M}$ gives $\min_{\mathcal{M}'} C \geq c^*$. So $\min_{\mathcal{M}'} C = c^*$, attained by M^* , and $\mathcal{M}' \neq \emptyset$ — indeed the incumbent witnesses it: every cell of M_J has $D \leq C(M_J) = \text{UB}$ by Proposition 1, so $M_J \in \mathcal{M}'$. (The code-level absence of a spurious “no valid root row” error is the Corollary, part (b).) $\square$

Corollary 1 (bit-identical result). *Let M_{ret} be the matching `extract_cell` returns on the unpruned instance. `extract_cell` contracts per pending-signature with a validity-blind minimum and runs `check_rules` only on the final list, so M_{ret} is the cheapest valid matching among the surviving signature representatives — not provably c^* on its own. Assume therefore the cross-validation invariant $C(M_{\text{ret}}) = C(\text{cell}) \leq C(\text{extract_join}) = \text{UB}$ (gated 384/384).*

- (a) M_{ret} is valid with $C(M_{\text{ret}}) \leq \text{UB}$, so Proposition 1 applies: every cell (m, v) it uses has $D[m][v] \leq C(M_{\text{ret}}) \leq \text{UB}$, hence $(m, v) \notin \text{blocked}$ — M_{ret} uses no blocked cell and all its cells survive.
- (b) M_{ret} is the cheapest matching of its own final pending-signature (else a cheaper same-signature matching would have been the unpruned representative); removing blocked cells — none of them M_{ret} 's — creates no new rows and no cheaper same-signature matching, so M_{ret} remains the pruned finals representative of its signature and, being valid, surfaces. The pruned run returns the same M_{ret} , with no spurious error.

Remark (reconvergent residual). *Proposition 1 bounds only valid matchings, so one case is not excluded by this proof: an invalid, blocked-cell-using finals candidate with cost $\leq C(M_{\text{ret}})$ sharing a post-early-discharge signature with a valid resurfacing row — which requires an early-discharged (reconvergent) merge. On **tree** sources (the hourglass deployment, where early discharge never fires, so every blocked merge cell stays in the final signature) this cannot arise and bit-parity is exact up to exact-cost string ties; on reconvergent DAGs the residual is covered by the empirical parity gate.*

The proof rests entirely on Proposition 1, whose only non-elementary ingredient is that D is the minimum-over-configurations above (the established semantics of `forward()`); the feasibility of $\widehat{M}$ and $\gamma \leq 1$ do the rest.

Why $D + B - E$ is *not* a valid bound

Remark. *The refuted first draft used $\text{LB}'(a, v) = D[a][v] + B[a][v] - E(a, v)$, with B the symmetric backward table (so $B[a][v]$ bounds the downstream cost from (a, v)). D accounts a 's run up to v ; B accounts it from v . When a 's true run in M is multi-cell (any $\alpha < 1$ coverage, with v interior), the sum charges the coverage of a 's run twice, and $-E(a, v)$ cancels only one cell's overlap. Thus $\text{LB}'(a, v)$ can exceed $C(M^*)$ at the entry pivot — violating Proposition 1 — so an optimal cell may satisfy $\text{LB}' > \text{UB}$, part (1) of Theorem 1 fails, M^* is blocked, $\mathcal{M}'$ becomes infeasible, and the extraction raises a spurious error (observed 15/7997 in an adversarial sweep on articulation merges). D alone carries no downstream term, so no such double-count arises, and Proposition 1 is a theorem for it. A sound tightening, if D -only is ever too loose, is $\text{LB}(m, v) = \max(D[m][v], B[m][v])$: the maximum of two independently valid lower bounds is a valid lower bound, with no gluing.*